

PlayerPose Module

1. Overview

The **PlayerPose** module is responsible for estimating and visualizing the body pose of players detected in video sequences. It leverages a YOLO keypoint detection model to extract player joints, map them back to the full frame, and draw skeletons for analysis of player movement and biomechanics.

2. Dependencies

- **Ultralytics YOLO:** for player keypoint detection.
- **OpenCV (cv2):** for image manipulation, cropping, and drawing skeletons.
- **NumPy:** for mathematical operations on keypoints.
- **Pickle:** for storing and loading precomputed keypoint results (stubs).

3. Class: PlayerPose

Initialization

`PlayerPose(model_path: str, kp_radius: int = 3, conf_threshold: float = 0.15, pad: int = 20, imsz: int = 640)`

- **Parameters:**
 1. `model_path`: Path to the YOLO pose estimation model.
 2. `kp_radius`: Radius (in pixels) for drawing keypoints.
 3. `conf_threshold`: Minimum confidence threshold for accepting keypoints.
 4. `pad`: Padding added when cropping around detected players.
 5. `imsz`: Input size for the YOLO model.

Methods

Crop Method

`_crop_with_padding(frame, bbox, pad=20)`

- **Purpose:** Crop a player region from a frame with padding.
- **Input:** Frame and bounding box coordinates.
- **Output:** Cropped image and bounding box coordinates.

Keypoint Mapping

`_map_keypoints(kps, crop_box, crop_shape)`

- **Purpose:** Map normalized YOLO keypoints from the crop back to the original image space.
- **Input:** Keypoints, crop bounding box, crop dimensions.
- **Output:** Array of mapped keypoints with confidence values.

Keypoint Extraction

`_extract_keypoints(res)`

- **Purpose:** Extract keypoints from YOLO result tensors.
- **Output:** List of NumPy arrays containing [x, y, confidence].

Pose Calculation

`calc_estimate_pose(video, player_dt)`

- **Purpose:** Estimate poses frame by frame using detected player bounding boxes.
- **Input:** Video frames, player detection dictionary per frame.
- **Output:** List of dictionaries mapping player IDs to keypoints per frame.

Pose Estimation (with stubs)

`estimate_pose(video, player_dt, read_stub=False, stub_path=None)`

- **Purpose:** Main interface for estimating poses. Supports loading/saving from pickle stubs.
- **Parameters:**
 - `read_stub`: If True, loads keypoints from pickle.
 - `stub_path`: Path to save/load keypoints.
- **Output:** Player pose data across all frames.

Pose Drawing

`draw_pose(video, player_pose_dt, color=(255, 255, 0))`

- **Purpose:** Draw keypoints and skeleton connections on each frame.
- **Input:** Video frames and corresponding pose data.
- **Output:** Annotated video frames.

4. Notes

- This module requires a YOLO pose model like `yolo11x-pose.pt`
- It integrates directly after PlayerTracking, which provides bounding boxes of detected players.
- The results can be combined with CourtDetection and BallTracking for biomechanical and tactical analysis.
- Pickle-based stubs are useful to avoid recomputation during repeated experiments.
- The output (pose-annotated video or raw keypoint data) can be used for performance metrics, injury prevention studies, or strategy analysis.